

Case Study: Online Orders Management System

Scenario

An **e-commerce platform** stores customer orders in a MongoDB collection called **orders**.

Each document contains:

- customerName
- product
- category
- quantity
- price
- totalAmount

Aggregate Functions - min, max, sum, avg, count

Groupby, match, project

```
db.orders.insertMany([
  { customerName: "Asha", product: "Laptop", category: "Electronics", quantity: 1,
    price: 50000, totalAmount: 50000 },
  { customerName: "Ravi", product: "Mobile", category: "Electronics", quantity: 2,
    price: 20000, totalAmount: 40000 },
  { customerName: "Kiran", product: "Shoes", category: "Fashion", quantity: 3, price:
    2000, totalAmount: 6000 },
  { customerName: "Divya", product: "Watch", category: "Accessories", quantity: 2,
    price: 3000, totalAmount: 6000 },
  { customerName: "Manoj", product: "Table", category: "Furniture", quantity: 1,
    price: 10000, totalAmount: 10000 },
  { customerName: "Sneha", product: "Chair", category: "Furniture", quantity: 4, price:
    2500, totalAmount: 10000 },
  { customerName: "Arun", product: "Headphones", category: "Electronics", quantity:
    2, price: 1500, totalAmount: 3000 },
  { customerName: "Meena", product: "Bag", category: "Fashion", quantity: 2, price:
    1200, totalAmount: 2400 },
```

```
{ customerName: "Vijay", product: "TV", category: "Electronics", quantity: 1, price: 30000, totalAmount: 30000 },  
{ customerName: "Priya", product: "Dress", category: "Fashion", quantity: 5, price: 1000, totalAmount: 5000 }  
])
```

2. Aggregate Functions

◆ Find Minimum Order Amount

```
db.orders.aggregate([  
  { $group: { _id: null, minAmount: { $min: "$totalAmount" } } }  
])
```

Show more lines

◆ Find Maximum Order Amount

```
db.orders.aggregate([  
  { $group: { _id: null, maxAmount: { $max: "$totalAmount" } } }  
])
```

◆ Find Average Order Amount

```
db.orders.aggregate([  
  { $group: { _id: null, avgAmount: { $avg: "$totalAmount" } } }  
])
```

◆ Find Total Number of Orders (Count)

```
db.orders.aggregate([  
  { $group: { _id: null, totalOrders: { $sum: 1 } } }  
])
```

Show more lines

◆ Find Total Revenue (Sum)

```
db.orders.aggregate([
{ $group: { _id: null, totalRevenue: { $sum: "$totalAmount" } } }
])
```

Show more lines

3. Aggregation Pipeline Stages

◆ \$match (Filter Orders > 10000)

```
db.orders.aggregate([
{ $match: { totalAmount: { $gt: 10000 } } }
])
```

◆ \$project (Show Only Customer & Amount)

```
db.orders.aggregate([
{ $project: { customerName: 1, totalAmount: 1, _id: 0 } }
])
```

◆ \$limit (Limit Results)

```
db.orders.aggregate([
{ $limit: 5 }
])
```

Show more lines

◆ \$skip (Skip First 3 Orders)

```
db.orders.aggregate([
{ $skip: 3 }
])
```

◆ \$sort (Sort by Total Amount Descending)

```
db.orders.aggregate([
  { $sort: { totalAmount: -1 } }
])
```

◆ \$group (Group by Category with Total Revenue)

```
db.orders.aggregate([
  {
    $group: {
      _id: "$category",
      totalRevenue: { $sum: "$totalAmount" },
      avgAmount: { $avg: "$totalAmount" },
      maxAmount: { $max: "$totalAmount" },
      minAmount: { $min: "$totalAmount" },
      count: { $sum: 1 }
    }
  }
])
```

Using \$limit and \$skip in MongoDB Aggregate

◆ Example 1: Limit Results

MongoDB

```
db.orders.aggregate([
  { $limit: 5 }
])
```

Explanation

Returns **only first 5 documents**
Same as SQL LIMIT

SQL Equivalent

```
SELECT * FROM orders LIMIT 5;
```

◆ Example 2: Skip Documents

MongoDB

```
db.orders.aggregate([  
  { $skip: 3 }  
])
```

Explanation

Skips the **first 3 documents**

Starts displaying from 4th document

SQL Equivalent

```
SELECT * FROM orders OFFSET 3;
```

◆ Example 3: Combine \$skip + \$limit (Pagination)

MongoDB

```
db.orders.aggregate([  
  { $skip: 5 },  
  { $limit: 5 }  
])
```

Explanation

Skips first **5 documents**

Then shows **next 5 documents** 🖱️ Used for **pagination**

SQL Equivalent

```
SELECT * FROM orders LIMIT 5 OFFSET 5;
```

◆ Example 4: With Sorting + Pagination

MongoDB

```
db.orders.aggregate([
  { $sort: { totalAmount: -1 } },
  { $skip: 2 },
  { $limit: 3 }
])
```

Explanation

Sort by **totalAmount** (descending)

Skip first 2 results

Show next 3 results

SQL Equivalent

```
SELECT * FROM orders
ORDER BY totalAmount DESC
LIMIT 3 OFFSET 2;
```

Example 1: Total Revenue by Category (High Value Orders)

? Question

Find total revenue for each category where order amount is greater than 10,000, show only category and revenue, and sort results in descending order.

✓ MongoDB

```
db.orders.aggregate([
  { $match: { totalAmount: { $gt: 10000 } } },
  { $group: { _id: "$category", totalRevenue: { $sum: "$totalAmount" } } },
  { $project: { category: "$_id", totalRevenue: 1, _id: 0 } },
  { $sort: { totalRevenue: -1 } }
])
```

SQL

```
SELECT category, SUM(totalAmount) AS totalRevenue
FROM orders
WHERE totalAmount > 10000
GROUP BY category
ORDER BY totalRevenue DESC;
```

Explanation

`$match` → filter high-value orders

`$group` → group by category

`$project` → rename fields

`$sort` → sort results

Example 2: Average Price per Product (Top 3 Results)

? Question

Find average order amount per product, show top 3 products with highest average.

MongoDB

```
db.orders.aggregate([
  { $group: { _id: "$product", avgAmount: { $avg: "$totalAmount" } } },
  { $project: { product: "$_id", avgAmount: 1, _id: 0 } },
  { $sort: { avgAmount: -1 } },
  { $limit: 3 }
])
```

SQL

```
SELECT product, AVG(totalAmount) AS avgAmount
```

```
FROM orders
GROUP BY product
ORDER BY avgAmount DESC
LIMIT 3;
```

Explanation

- Group by product
- Calculate average
- Sort descending
- Limit top 3

Example 3: Customer Orders (Skip + Limit)

? Question

Display 3 customers after skipping first 2 records, showing only name and order amount sorted by amount.

MongoDB

```
db.orders.aggregate([
  { $project: { customerName: 1, totalAmount: 1, _id: 0 } },
  { $sort: { totalAmount: -1 } },
  { $skip: 2 },
  { $limit: 3 }
])
```

SQL

```
SELECT customerName, totalAmount
FROM orders
ORDER BY totalAmount DESC
LIMIT 3 OFFSET 2;
```

Explanation

`$project` → select fields

`$sort` → sort by amount

`$skip` → skip first 2

`$limit` → show next 3

Example 4: Count Orders per Category (Filtered)

? Question

Find number of orders per category where quantity is greater than 2, show only categories with more than 1 order, sorted ascending.

✓ MongoDB

```
db.orders.aggregate([
  { $match: { quantity: { $gt: 2 } } },
  { $group: { _id: "$category", count: { $sum: 1 } } },
  { $match: { count: { $gt: 1 } } },
  { $sort: { count: 1 } }
])
```

✓ SQL

```
SELECT category, COUNT(*) AS count
FROM orders
WHERE quantity > 2
GROUP BY category
HAVING COUNT(*) > 1
ORDER BY count ASC;
```

Explanation

`$match` → filter quantity

`$group` → count per category

second `$match` → acts like HAVING

`$sort` → ascending order

Example 5: Total Quantity by Brand (Pagination)

? Question

Find total quantity sold per brand, show only 2 results after skipping 1, sorted by quantity descending.

✓ MongoDB

```
db.orders.aggregate([
  { $group: { _id: "$brand", totalQty: { $sum: "$quantity" } } },
  { $sort: { totalQty: -1 } },
  { $skip: 1 },
  { $limit: 2 }
])
```

✓ SQL

```
SELECT brand, SUM(quantity) AS totalQty
FROM orders
GROUP BY brand
ORDER BY totalQty DESC
LIMIT 2 OFFSET 1;
```

Explanation

Group by brand
Sum quantity
Sort results
Apply skip + limit (pagination)

Example 1

? Question

Find total revenue by category for orders above 10,000 and display in descending order.

MongoDB

```
db.orders.aggregate([
  { $match: { totalAmount: { $gt: 10000 } } },
  { $group: { _id: "$category", totalRevenue: { $sum: "$totalAmount" } } },
  { $project: { category: "$_id", totalRevenue: 1, _id: 0 } },
  { $sort: { totalRevenue: -1 } }
])
```

SQL

```
SELECT category, SUM(totalAmount) AS totalRevenue
FROM orders
WHERE totalAmount > 10000
GROUP BY category
ORDER BY totalRevenue DESC;
```

Explanation

`$match` → filters high-value orders (WHERE)
`$group` → groups by category (GROUP BY)
`$project` → selects output fields (SELECT)
`$sort` → sorts the result (ORDER BY)

Example 2

? Question

Find top 3 products with highest average order amount.

✓ MongoDB

```
db.orders.aggregate([
  { $group: { _id: "$product", avgAmount: { $avg: "$totalAmount" } } },
  { $project: { product: "$_id", avgAmount: 1, _id: 0 } },
  { $sort: { avgAmount: -1 } },
  { $limit: 3 }
])
```

✓ SQL

```
SELECT product, AVG(totalAmount) AS avgAmount
FROM orders
GROUP BY product
ORDER BY avgAmount DESC
LIMIT 3;
```

Explanation

- Creates groups per product
- Calculates average
- Sorts highest first
- Limits to top 3

Example 3

Question

Display 4 orders after skipping first 2, showing only customer name and amount sorted by amount.

✓ MongoDB

```
db.orders.aggregate([
  { $project: { customerName: 1, totalAmount: 1, _id: 0 } },
  { $sort: { totalAmount: -1 } },
  { $skip: 2 },
  { $limit: 4 }
])
```



```
SELECT customerName, totalAmount
FROM orders
ORDER BY totalAmount DESC
LIMIT 4 OFFSET 2;
```

Explanation

`$project` → selects required columns

`$sort` → sorts data

`$skip` → skips first 2 rows (OFFSET)

`$limit` → returns next 4 rows

Example 4

? Question

Find categories having more than 2 orders (only for quantity > 1), sorted ascending.



```
db.orders.aggregate([
  { $match: { quantity: { $gt: 1 } } },
  { $group: { _id: "$category", count: { $sum: 1 } } },
  { $match: { count: { $gt: 2 } } },
  { $sort: { count: 1 } }
])
```



```
SELECT category, COUNT(*) AS count
FROM orders
WHERE quantity > 1
GROUP BY category
HAVING COUNT(*) > 2
ORDER BY count ASC;
```

Explanation

First `$match` → acts like WHERE
`$group` → counts per category
Second `$match` → acts like HAVING
`$sort` → ascending order

Example 5

? Question

Find total quantity sold per category, skip first result and show next 2 highest.

✓ MongoDB

```
db.orders.aggregate([
  { $group: { _id: "$category", totalQty: { $sum: "$quantity" } } },
  { $sort: { totalQty: -1 } },
  { $skip: 1 },
  { $limit: 2 }
])
```

✓ SQL

```
SELECT category, SUM(quantity) AS totalQty
FROM orders
GROUP BY category
ORDER BY totalQty DESC
LIMIT 2 OFFSET 1;
```

Explanation

Groups by category
Calculates total quantity
Sorts descending
Skips top result
Shows next 2 results

Example 6: High Spending Customers Analysis

Title

High Spending Customers (Grouped & Paginated)

? Question

Find customers whose total order amount is greater than 5,000, calculate total spending per customer, display only customer name and total spending, sort by highest spending, skip the top customer and show next 2.

✓ MongoDB Query

```
db.orders.aggregate([
  { $match: { totalAmount: { $gt: 5000 } } },
  { $group: { _id: "$customerName", totalSpent: { $sum: "$totalAmount" } } },
  { $project: { customerName: "$_id", totalSpent: 1, _id: 0 } },
  { $sort: { totalSpent: -1 } },
  { $skip: 1 },
  { $limit: 2 }
])
```

✓ SQL Query

```
SELECT customerName, SUM(totalAmount) AS totalSpent
FROM orders
```

```
WHERE totalAmount > 5000
GROUP BY customerName
ORDER BY totalSpent DESC
LIMIT 2 OFFSET 1;
```

Explanation

`$match` → filters only customers with orders > 5000 (**WHERE**)
`$group` → groups by customer and sums spending (**GROUP BY**)
`$project` → selects fields (**SELECT**)
`$sort` → highest spending first (**ORDER BY**)
`$skip` → skips top customer (**OFFSET**)
`$limit` → shows next 2 customers (**LIMIT**)

Example 7: Category-wise Profit (Filtered with HAVING)

Title

Category Revenue with Minimum Threshold

Question

Find total revenue per category where price is greater than 1000, display categories having total revenue greater than 20,000, sort in descending order and show only top 3.



MongoDB Query

```
db.orders.aggregate([
  { $match: { price: { $gt: 1000 } } },
  { $group: { _id: "$category", totalRevenue: { $sum: "$totalAmount" } } },
  { $match: { totalRevenue: { $gt: 20000 } } },
  { $project: { category: "$_id", totalRevenue: 1, _id: 0 } },
  { $sort: { totalRevenue: -1 } },
  { $limit: 3 }
```

1)

SQL Query

```
SELECT category, SUM(totalAmount) AS totalRevenue
FROM orders
WHERE price > 1000
GROUP BY category
HAVING SUM(totalAmount) > 20000
ORDER BY totalRevenue DESC
LIMIT 3;
```

Explanation

First `$match` → filters records by price (**WHERE**)

`$group` → calculates total revenue per category (**GROUP BY**)

Second `$match` → filters aggregated results (**HAVING**)

`$project` → selects output fields (**SELECT**)

`$sort` → orders results (**ORDER BY**)

`$limit` → restricts to top 3 (**LIMIT**)

Example 8: Top Selling Products by Quantity

Title

Top Products Based on Quantity Sold (With Pagination)

Question

Find products where quantity is greater than 1, calculate total quantity sold per product, display only product name and total quantity, sort in descending order, skip 1 product and show next 3.

MongoDB Query

```
db.orders.aggregate([
  { $match: { quantity: { $gt: 1 } } },
  { $group: { _id: "$product", totalQty: { $sum: "$quantity" } } },
  { $project: { product: "$_id", totalQty: 1, _id: 0 } },
  { $sort: { totalQty: -1 } },
  { $skip: 1 },
  { $limit: 3 }
])
```



SQL Query

```
SELECT product, SUM(quantity) AS totalQty
FROM orders
WHERE quantity > 1
GROUP BY product
ORDER BY totalQty DESC
LIMIT 3 OFFSET 1;
```

Explanation

`$match` → filters products with quantity > 1 (**WHERE**)

`$group` → sums quantity per product (**GROUP BY**)

`$project` → selects and renames fields (**SELECT**)

`$sort` → orders products by quantity (**ORDER BY**)

`$skip` → skips top product (**OFFSET**)

`$limit` → shows next 3 products (**LIMIT**)

Example 9: Low Revenue Categories Analysis

Title

Categories with Moderate Revenue (Filtered & Limited)

Question

Find categories where total order amount is greater than 1000, calculate total revenue per category, include only categories where revenue is greater than 5000, display category and revenue, sort ascending, and show top 2.

MongoDB Query

```
db.orders.aggregate([
  { $match: { totalAmount: { $gt: 1000 } } },
  { $group: { _id: "$category", totalRevenue: { $sum: "$totalAmount" } } },
  { $match: { totalRevenue: { $gt: 5000 } } },
  { $project: { category: "$_id", totalRevenue: 1, _id: 0 } },
  { $sort: { totalRevenue: 1 } },
  { $limit: 2 }
])
```

SQL Query

```
SELECT category, SUM(totalAmount) AS totalRevenue
FROM orders
WHERE totalAmount > 1000
GROUP BY category
HAVING SUM(totalAmount) > 5000
ORDER BY totalRevenue ASC
LIMIT 2;
```

Explanation

First `$match` → filters records based on amount (**WHERE**)
`$group` → groups by category and computes revenue (**GROUP BY**)
Second `$match` → filters grouped result (**HAVING**)
`$project` → selects required fields (**SELECT**)
`$sort` → sorts in ascending order (**ORDER BY**)
`$limit` → restricts to 2 results (**LIMIT**)